DQN

김영은

DQN (Deep Q-Network)

상태 s를 입력으로 받고, 각 행동 a에 대한 Q값 $Q(s, a; \theta)$ 를 신경망 하나로 근사하는
value-based, model-free, off-policy 알고리즘

원래의 Q-learning 테이블을 고차원 상태(이미지 등)에서도 쓰게 해 주는 버전

Q-learning

목표는 최적 Q함수 $Q^*(s,a)$

$$Q^*(s,a) = \mathbb{E}\left[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a\right]$$

반복적 update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big)$$

$r_t + \gamma \max_{a'} Q(s_{t+1}, a')$: 타깃(목표값)

$Q(s_t, a_t)$: 현재 추정 (차이가 TD error)

테이블 방식에서는 Q(s,a)를 배열로 들고 있었지만,
상태가 이미지 같은 고차원일 때는 배열로는 불가능 → 신경망으로 근사 → DQN

Main Architecture - neural network

- 입력: 상태 s (예: $84 \times 84 \times 4$ 프레임 스택된 이미지)
- CNN 또는 MLP를 통과
- 출력: 가능한 모든 행동에 대한 Q값 벡터 $[Q(s, a_1), \ldots, Q(s, a_{|A|})]$

즉, 행동 하나마다 네트워크를 따로 만드는 게 아니라, 네트워크 하나가 한 번에 모든 행동의 Q값을 뽑음

Main Architectire - loss function

경험샘플 $(s_t, a_t, r_t, s_{t+1}, done)$에 대해

$$y_t = \begin{cases} r_t, & \text{if done} \\ r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-), & \text{otherwise} \end{cases}$$

θ : 현재 Q-네트워크 파라미터
θ^- : 타깃 네트워크 파라미터

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y_t - Q(s_t, a_t; \theta)\right)^2\right]$$ (gradient descent / Adam 등으로 최적화)

DL + Q-learning

딥러닝 + Q-learning을 그냥 섞으면 거의 무조건 발산

Experience Replay

에이전트가 환경과 상호작용하며 얻은 transition을 D라는 버퍼에 저장:

$$(s_t, a_t, r_t, s_{t+1}, done)$$

→ 학습할 때는 이 버퍼에서 무작위로 미니배치 샘플링해서 업데이트

Experience Replay

1. 데이터 상관성 감소
- 순차적으로 나온 샘플(시계열)을 그대로 학습하면 샘플 간 상관성이 커서 gradient가 불안정해짐
- 랜덤 샘플링으로 i. i. d. 에 좀 더 가깝게 만듦 → 학습 안정

2. 데이터 재사용 (sample efficiency 증가)
- 한 번 경험한 transition을 여러 번 재사용 가능 → 표본 효율성 증가

3. Off-policy 학습 가능
- 현재 정책이 아니라, 예전에 탐험하면서 모은 데이터들도 계속 사용 가능

Experience Replay

메모리 사용량 큼 (대규모 버퍼 필요)

- 환경이 빠르게 바뀌거나 non-stationary하면 예전 샘플이 “오염된” 데이터가 됨
- 실시간/온라인 환경에서는 큰 버퍼가 비현실적일 수 있음

Target Network

Q-러닝 target: $\max_{a'} Q(s_{t+1}, a')$

- 만약 이 Q를 같은 네트워크 θ로 계산하면,
- 타깃(y)도, 예측(Q)도 동시에 계속 변함 $\theta^- \leftarrow \theta$
- "움직이는 타깃을 쫓는" 꼴이라 학습이 매우 불안정해짐

그래서 두 개의 네트워크를 둠

- Online Q-network: $Q(s, a; \theta)$ — 학습 대상
- Target Q-network: $Q(s, a; \theta^-)$ — 타깃 계산용, 잠시 고정

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-)$$

$$\theta^- \leftarrow \theta$$

Target Network

효과:

- 타깃이 짧은 기간 동안 “고정”되어,

비선형 함수 근사기(신경망) + 부트스트래핑 조합의 폭주를 많이 완화

Target Network

단점:

- 아직도 근본적으로는 “동일 모델 구조”로 타깃을 만드는 자기참조 구조
- 완벽한 안정성은 아님 → Double DQN, C_{51}, Rainbow 등 추가 기법 필요

DQN Algorithm

1. Q-네트워크 파라미터 θ 랜덤 초기화
2. 타깃 네트워크 파라미터 $\theta^{-} \leftarrow \theta$
3. Replay Buffer D 초기화
4. 반복

Pros

(1) 고차원 상태공간에서 “테이블 Q-러닝”의 한계를 깨부숨
- 상태공간이 엄청 커도 (심지어 연속) Q-러닝을 쓸 수 있게 만든 첫 실질적 성공

(2) Off-policy라 데이터 재사용 가능 → 표본 효율성이 Tabular보다 훨씬 나음
- Replay Buffer에서 옛날 transition도 계속 활용 → 특히 시뮬레이터 환경에서 유리
- 한 번의 환경 상호작용을 여러 번 학습에 활용

(3) 구현 난이도가 상대적으로 낮은 편
- “값 기반 RL + supervised regression” 구조

→ 딥러닝 프레임워크만 다룰 줄 알면 구현 자체는 명확
- 정책경사 계열(PG, Actor-Critic)보다 수학적으로도 직관적

(4) 향후 수많은 확장 기법의 “기본 베이스라인”
- Double DQN, Dueling Network, Rainbow 등 대부분이 DQN 구조를 베이스로 발전
- RL 논문, 프로젝트에서 여전히 기준선 역할

Cons

1) 여전히 매우 불안정하고 하이퍼파라미터에 민감

- 학습률, ε-스케줄, 리플레이 버퍼 크기, 타깃 업데이트 주기 등 조금만 바뀌어도

성능 크게 요동

- 동일한 세팅에서도 랜덤 시드에 따라 결과가 극단적으로 달라질 수 있음

(2) Overestimation Bias (Q값 과대추정)

타깃에서 $\max Q(s', a')$를 사용할 때, 같은 네트워크가 행동을 고르고, 그 값도 평가하기 때문에 노이즈 + max 연산 때문에 Q값이 과대평가되는 경향

- 이 때문에 학습이 더 불안정해지고, 잘못된 행동을 과신하게 됨
- 이를 줄이려고 나온 것이 Double DQN

(행동 선택은 online network, 타깃 평가는 target network로 분리)

Afterward?

DQN

Q-learning + 신경망

Q값을 신경망을 통해 예측

$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

☆ ↓ 과대평가 발생 부분

손실함수로 학습 $L(\theta) = E[r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta)]^2$

(θ : 현재 network param, θ^- : target network param)

DDQN

DQN의 논리적 해석의 단점 → Q값 계산을 다른 신경망이 하도록 변경

(Action 고르는 신경망 ≠ 정확 Q 매기는 신경망)

$y^{DDQN} = r + \gamma Q(s', \arg\max_{a'} Q(s',a';\theta); \theta^-)$

$L(\theta) = E[(y^{DDQN} - Q(s,a;\theta))^2]$

Dueling DQN

Q값 계산 [State value → 이 상황이 얼마나 좋은가

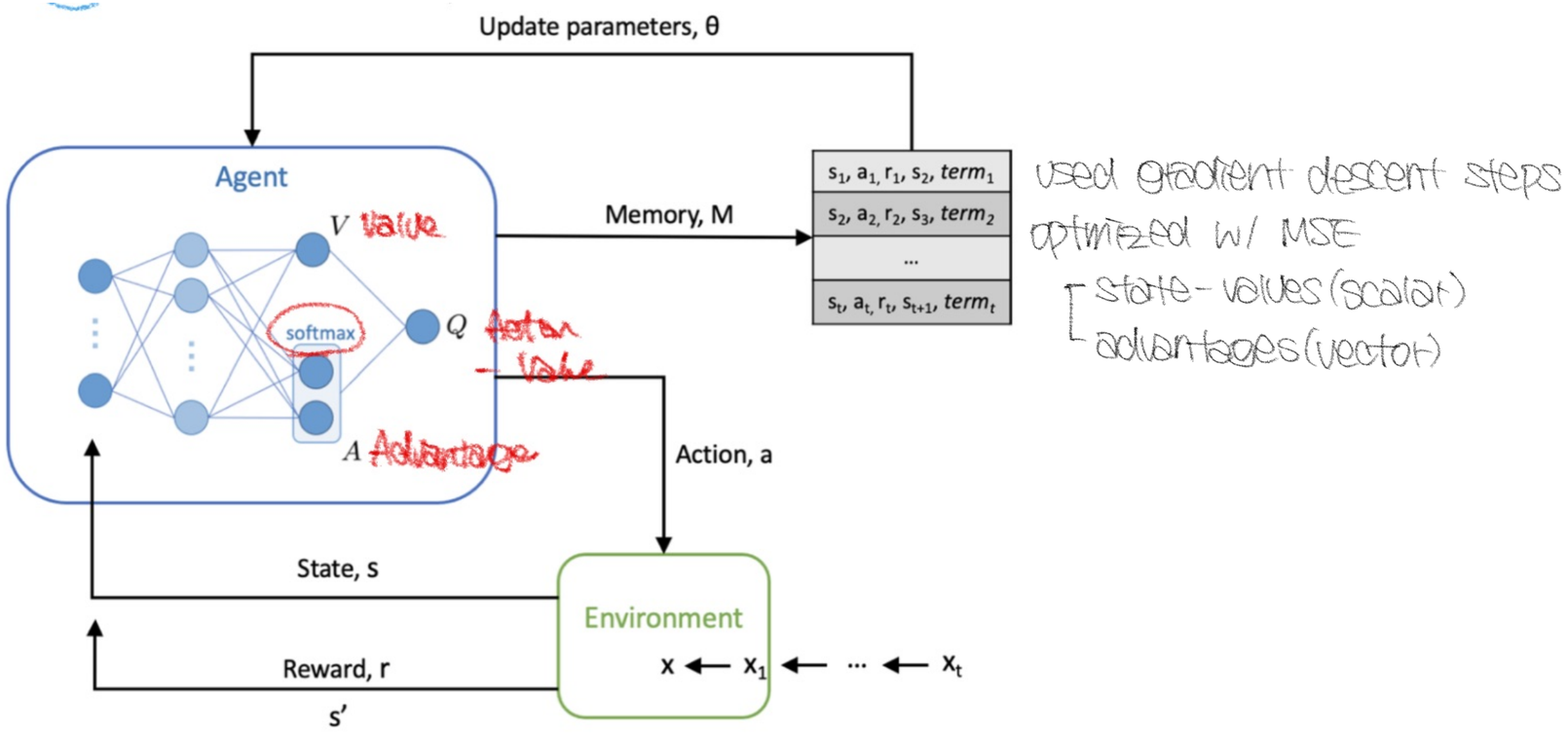
Action advantage → 이 상황에서 이 행동이 얼마나 좋은가] $Q(s,a)$

$Q(s,a) = V(s) + A(s,a)$

e.g., 이미 미로의 출구 앞에 있다면, 행동에 크게 상관없이 좋은 상태임

$= V(s) + (A(s,a) - \frac{1}{|A|}\sum_{a'} A(s,a'))$

Afterward?



기존의 DQN은 max때문에 optimal policy를 정확히 계산하기 어려움
→ DDQN은 selection과 evaluation 파트를 분리
current Q-network – select actions
target Q-network – estimate select actions' value

DQN